ADR 0008: Commerce lives in Core; the vendor-admin service is the seller back-office

Status	Accepted by the team (2026-10-20). Supersedes the "orders in the marketplace" choice in the first version of ADR 0007
Date	2026-10-20
Related	`architecture.md` sections 2 and 5, `08-service-split.md`, ADR 0006, ADR 0007

Context

The first two-service draft gave the FastAPI service (then called "marketplace") the orders, the order state machine, stock and payments, including Razorpay. The team pointed out that this is wrong for an e-commerce platform: checkout and payment belong to the customer's buying path, and a seller back-office should not hold the payment gateway or be on the critical path of every purchase.

Options

1. **Commerce in Core** (cart, checkout, orders, stock, payments, refunds, returns). The vendor-admin service is the back-office and acts on orders and stock only by asking Core.
2. **A third "commerce" service.** Cleanest boundaries, but three services for three engineers by Oct 30.
3. **Orders in the vendor-admin service, payments in Core.** Splits an order from its payment; hard to keep consistent.

Decision

Option 1.

- Core owns carts, inventory and stock reservations, orders and the order state machine, payments, refunds, deliveries and returns, and the Razorpay integration (create order, hosted checkout, signed webhook, refund).
- **Stock is owned by Core.** The vendor sets `on_hand` through the vendor-admin service, which sends it to Core. Checkout reserves stock atomically in Postgres (`reserved` incremented after a row lock in id order), ship commits it, cancel or expiry releases it.
- **Checkout is one Core transaction** and never calls the vendor-admin service. Price and commission are read from Core's versioned catalog copy (published by the vendor-admin service) and snapshotted on the order together with the catalog versions used.
- The vendor-admin service keeps a **read-only copy of orders** (built from events) for the portal and for statements. Fulfilment actions (confirm, ship, cancel, deliver, payment collected, refund, return decision) are **commands** from the vendor-admin service to Core, which changes the order, records the actor, and sends the customer push.
- The vendor-admin service holds no payment credentials and never changes money, stock or order state.
- The online payment switch (`online_payments_enabled`) is a Core setting; the app learns it from Core's public config.

Consequences

- Checkout and payments work while the vendor-admin service is down; customers are not affected by back-office outages.
- Core grows and is the largest track; the vendor-admin service is lighter. Tracks are rebalanced (`architecture.md` section 14).
- The "oversell the last item and let the vendor cancel one" risk of the first draft disappears.
- Admin order operations cannot be plain SQLAdmin (it edits only its own database); they are small portal admin pages that call the vendor-admin API, which sends commands to Core. SQLAdmin remains for vendors, products, users and audit.
- Price is mastered in the vendor-admin service and reaches Core by event, so there is a short staleness window; orders store the version they were priced at.
- PCI scope stays minimal because card data is entered only on Razorpay's hosted checkout.
- Database ownership moves: orders, payments, refunds, deliveries, returns and the new inventory and reservation tables are in `shazlo_core`; the vendor-admin database gets the `vendor_orders` copy, commission history and payouts.
- Core now emits the order and payment events; the vendor-admin service emits product, vendor and commission events.

Revisit when

- Core's commerce code needs a different team or release rhythm: extract it as a third service (the module boundary `orders` inside Core is kept clean for this).
- Razorpay Route (automatic vendor splits) is adopted: payout logic may need a dedicated module or service.